

Thyroid_Disease_Prediction

Table Content

- [Library call](#)
- [EDA](#)
 - [Age Analysis](#)
 - [Gender Analysis](#)
 - [Focality Analysis](#)
 - [Stage Analysis](#)
 - [Hx-Radiothreapy Analysis](#)
 - [Adenopathy Analysis](#)
 - [Pathology Analysis](#)
 - [Physical Examination Analysis](#)
 - [Tumor Analysis](#)
 - [Nodal Analysis](#)
 - [Metastasis Analysis](#)
 - [Data prepossessing](#)
 - [split](#)
 - [prepossesse categorical columns](#)
 - [prepocssesse numeric columns](#)
- [Modeling](#)
 - [cross-validation](#)
 - [Analysis a model performance](#)
 - [choose our best model for this dataset](#)
 - [Optimize our Final Model](#)
 - [Final model with parameter and performance](#)
- [Create a single output](#)
 - [create a single input and predict stepwise](#)
 - [create single predict function](#)
- [Save and Load the model](#)
 - [save](#)
 - [load](#)
 - [performance](#)

About Dataset

This data set contains 13 clinicopathologic features aiming to predict recurrence of well differentiated thyroid cancer. The data set was collected in duration of 15 years and each patient was followed for at least 10 years.

Source

The data was procured from thyroid disease datasets provided by the UCI Machine Learning Repository.

Content

The size for the file featured within this Kaggle dataset is shown below — along with a list of attributes, and their description summaries:

Age: The age of the patient at the time of diagnosis or treatment.

Gender: The gender of the patient (male or female).

Smoking: Whether the patient is a smoker or not.

Hx Smoking: Smoking history of the patient (e.g., whether they have ever smoked).

Hx Radiotherapy: History of radiotherapy treatment for any condition.

Thyroid Function: The status of thyroid function, possibly indicating if there are any abnormalities.

Physical Examination: Findings from a physical examination of the patient, which may include palpation of the thyroid gland and surrounding structures.

Adenopathy: Presence or absence of enlarged lymph nodes (adenopathy) in the neck region.

Pathology: Specific types of thyroid cancer as determined by pathology examination of biopsy samples.

Focality: Whether the cancer is unifocal (limited to one location) or multifocal (present in multiple locations).

Risk: The risk category of the cancer based on various factors, such as tumor size, extent of spread, and histological type.

T: Tumor classification based on its size and extent of invasion into nearby structures.

N: Nodal classification indicating the involvement of lymph nodes.

M: Metastasis classification indicating the presence or absence of distant metastases.

Stage: The overall stage of the cancer, typically determined by combining T, N, and M classifications.

Response: Response to treatment, indicating whether the cancer responded positively,

Library call

```
In [7]: import pandas as pd
import numpy as np
import seaborn as sns
import plotly.express as px
import matplotlib.pyplot as plt
from sklearn.preprocessing import OneHotEncoder,MinMaxScaler,LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression,SGDClassifier
from sklearn.ensemble import RandomForestClassifier,AdaBoostClassifier,GradientBoostingClassifier
from sklearn.svm import SVC
from xgboost import XGBRFClassifier
from sklearn.metrics import classification_report,confusion_matrix,ConfusionMatrixDisplay
import joblib
```

```
In [8]: df = pd.read_csv("C:\\Users\\jemin kamani\\Downloads\\Thyroid_Diff.csv",low_memory=False)
```

```
In [9]: df
```

Out[9]:

	Age	Gender	Smoking	Hx Smoking	Hx Radiotherapy	Thyroid Function	Physical Examination	Adenopathy
0	27	F	No	No	No	Euthyroid	Single nodular goiter-left	
1	34	F	No	Yes	No	Euthyroid	Multinodular goiter	
2	30	F	No	No	No	Euthyroid	Single nodular goiter-right	
3	62	F	No	No	No	Euthyroid	Single nodular goiter-right	
4	62	F	No	No	No	Euthyroid	Multinodular goiter	
...	...	...	...	...	...	...	...	...
378	72	M	Yes	Yes	Yes	Euthyroid	Single nodular goiter-right	F
379	81	M	Yes	No	Yes	Euthyroid	Multinodular goiter	Exter
380	72	M	Yes	Yes	No	Euthyroid	Multinodular goiter	Bila
381	61	M	Yes	Yes	Yes	Clinical Hyperthyroidism	Multinodular goiter	Exter
382	67	M	Yes	No	No	Euthyroid	Multinodular goiter	Bila

383 rows × 17 columns

In [10]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 383 entries, 0 to 382
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   383 non-null    int64
1   Gender                               383 non-null    object
2   Smoking                              383 non-null    object
3   Hx Smoking                           383 non-null    object
4   Hx Radiothreapy                       383 non-null    object
5   Thyroid Function                      383 non-null    object
6   Physical Examination                 383 non-null    object
7   Adenopathy                           383 non-null    object
8   Pathology                            383 non-null    object
9   Focality                             383 non-null    object
10  Risk                                  383 non-null    object
11  T                                     383 non-null    object
12  N                                     383 non-null    object
13  M                                     383 non-null    object
14  Stage                                383 non-null    object
15  Response                             383 non-null    object
16  Recurred                             383 non-null    object
dtypes: int64(1), object(16)
memory usage: 51.0+ KB
```

In [11]: `# rename`
`df.rename(columns={'T':'Tumor','N':'Nodal','M':'Metastasis'},inplace=True)`

In [12]: `df.columns`

```
Out[12]: Index(['Age', 'Gender', 'Smoking', 'Hx Smoking', 'Hx Radiothreapy',
                'Thyroid Function', 'Physical Examination', 'Adenopathy', 'Pathology',
                'Focality', 'Risk', 'Tumor', 'Nodal', 'Metastasis', 'Stage', 'Response',
                'Recurred'],
              dtype='object')
```

In [13]: `df`

Out[13]:

	Age	Gender	Smoking	Hx Smoking	Hx Radiothreapy	Thyroid Function	Physical Examination	Adenop:
0	27	F	No	No	No	Euthyroid	Single nodular goiter-left	
1	34	F	No	Yes	No	Euthyroid	Multinodular goiter	
2	30	F	No	No	No	Euthyroid	Single nodular goiter-right	
3	62	F	No	No	No	Euthyroid	Single nodular goiter-right	
4	62	F	No	No	No	Euthyroid	Multinodular goiter	
...	...	...	...	...	...	...	...	...
378	72	M	Yes	Yes	Yes	Euthyroid	Single nodular goiter-right	F
379	81	M	Yes	No	Yes	Euthyroid	Multinodular goiter	Exter
380	72	M	Yes	Yes	No	Euthyroid	Multinodular goiter	Bila
381	61	M	Yes	Yes	Yes	Clinical Hyperthyroidism	Multinodular goiter	Exter
382	67	M	Yes	No	No	Euthyroid	Multinodular goiter	Bila

383 rows × 17 columns

In [14]: `df.isnull().sum()`

```
Out[14]: Age                0
Gender                0
Smoking              0
Hx Smoking           0
Hx Radiothreapy      0
Thyroid Function     0
Physical Examination  0
Adenopathy           0
Pathology            0
Focality             0
Risk                 0
Tumor                0
Nodal                0
Metastasis           0
Stage                0
Response             0
Recurred             0
dtype: int64
```

```
In [15]: df.nunique()
```

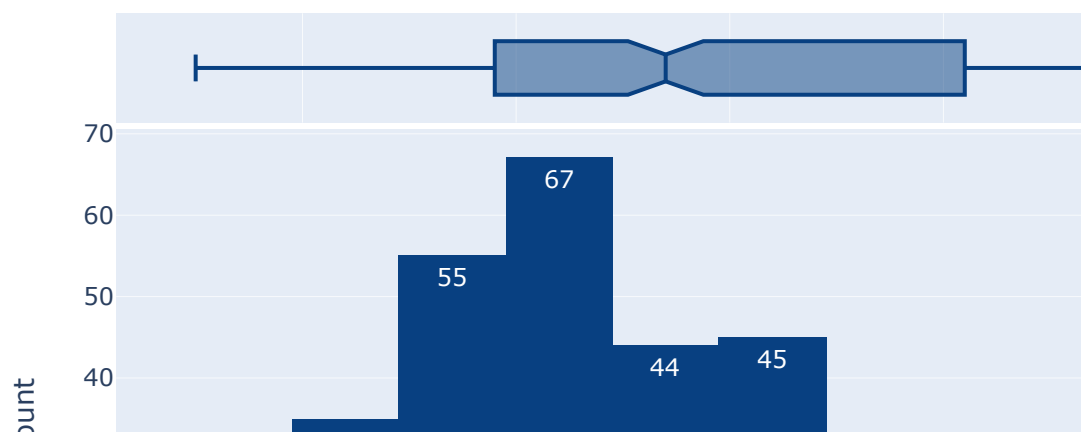
```
Out[15]: Age                65  
         Gender              2  
         Smoking            2  
         Hx Smoking         2  
         Hx Radiothreapy    2  
         Thyroid Function   5  
         Physical Examination 5  
         Adenopathy         6  
         Pathology          4  
         Focality           2  
         Risk               3  
         Tumor              7  
         Nodal              3  
         Metastasis         2  
         Stage              5  
         Response           4  
         Recurred           2  
         dtype: int64
```

EDA

Age Analysis

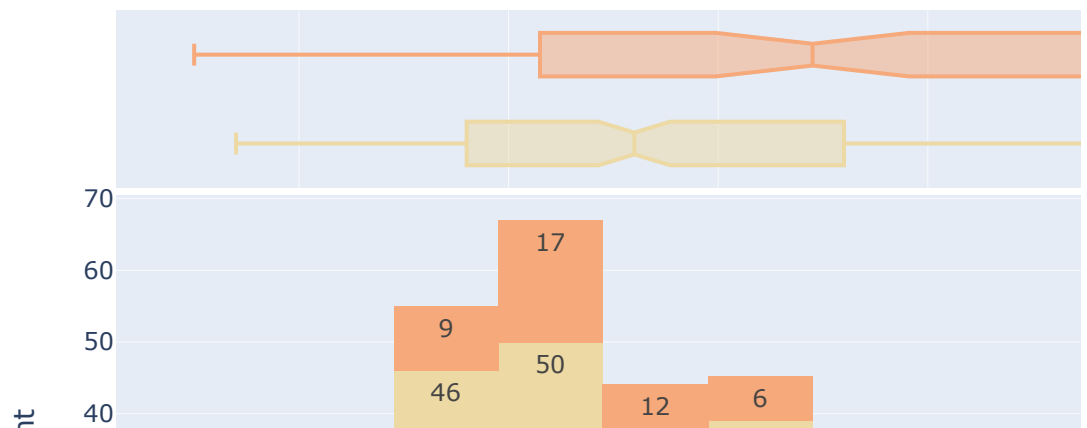
```
In [16]: fig = px.histogram(data_frame=df,  
                             x = 'Age',  
                             nbins=20,marginal='box',  
                             color_discrete_sequence=px.colors.sequential.GnBu_r,  
                             text_auto=True,title='The Distribution of Age')  
fig.update_layout()  
fig.show()
```

The Distribution of Age



```
In [17]: fig = px.histogram(data_frame=df,
                             x='Age',
                             nbins=20, marginal='box',
                             color='Recurred',
                             color_discrete_sequence=px.colors.sequential.Agsunset_r,
                             text_auto=True, title='The Distribution of Recurred base
fig.update_layout()
fig.show()
```

The Distribution of Recurred based on Age



Gender Analysis

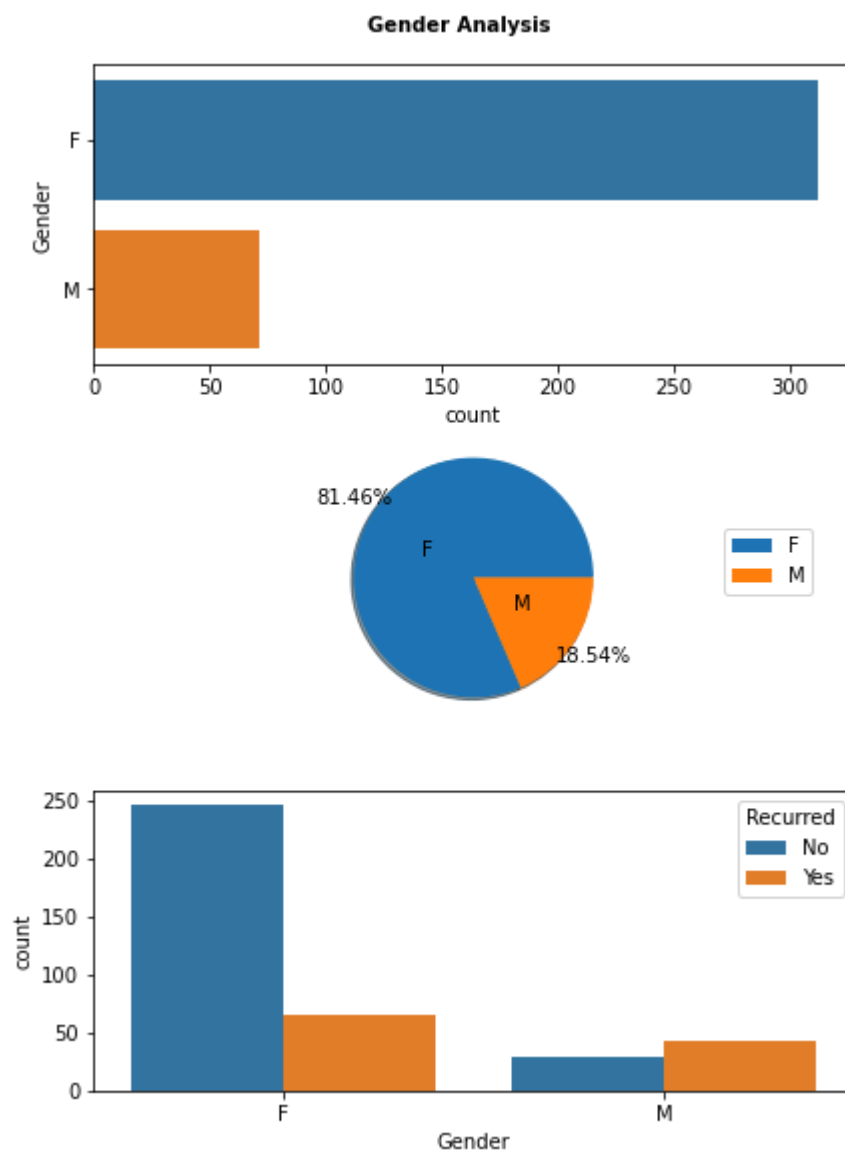
```
In [18]: fig, ax = plt.subplots(3, 1, figsize=(6, 8))
fig.suptitle('Gender Analysis', fontsize=10, fontweight='bold')
plt.tight_layout()

count = df['Gender'].value_counts()

labels = df['Gender'].value_counts().index.tolist()

#Top ax
sns.countplot(y="Gender", data=df, ax=ax[0])
#middle ax
ax[1].pie(count, autopct='%.2f%%', labels=labels, shadow=True, pctdistance=
ax[1].legend(bbox_to_anchor=(1, 1), loc=2, borderaxespad=5)

# Bottom ax
sns.countplot(x='Gender', hue='Recurred', data=df, ax=ax[2])
plt.show()
```



Focality Analysis

```
In [19]: fig, ax = plt.subplots(3, 1, figsize=(6, 8))
fig.suptitle('Focality Analysis', fontsize=20, fontweight='bold')
plt.tight_layout()

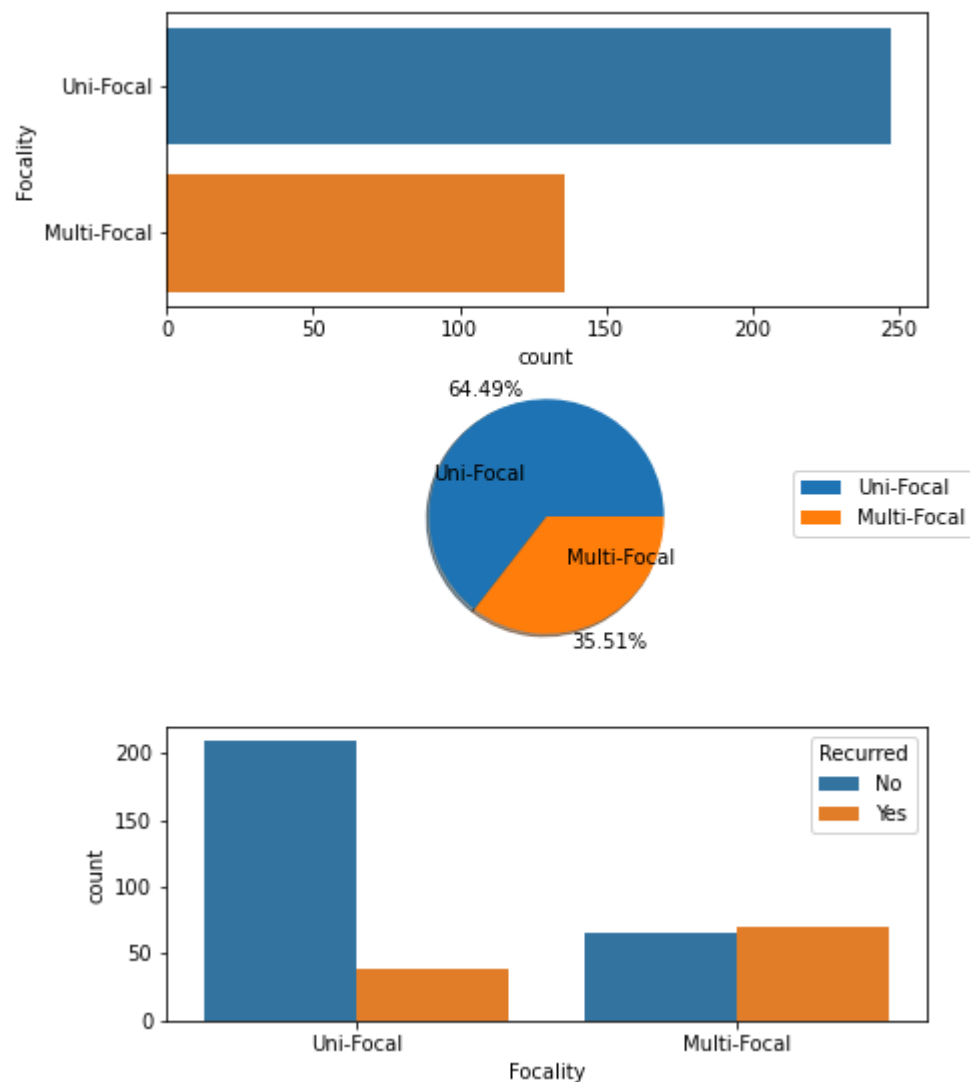
count = df['Focality'].value_counts()

labels = df['Focality'].value_counts().index.tolist()

#Top ax
sns.countplot(y="Focality", data=df, ax=ax[0])
#middle ax
ax[1].pie(count, autopct='%0.2f%%', labels=labels, shadow=True, pctdistance=0.8)
ax[1].legend(bbox_to_anchor=(1, 1), loc=2, borderaxespad=5)

# Bottom ax
sns.countplot(x='Focality', hue='Recurred', data=df, ax=ax[2])
plt.show()
```

Focality Analysis



Stage Analysis

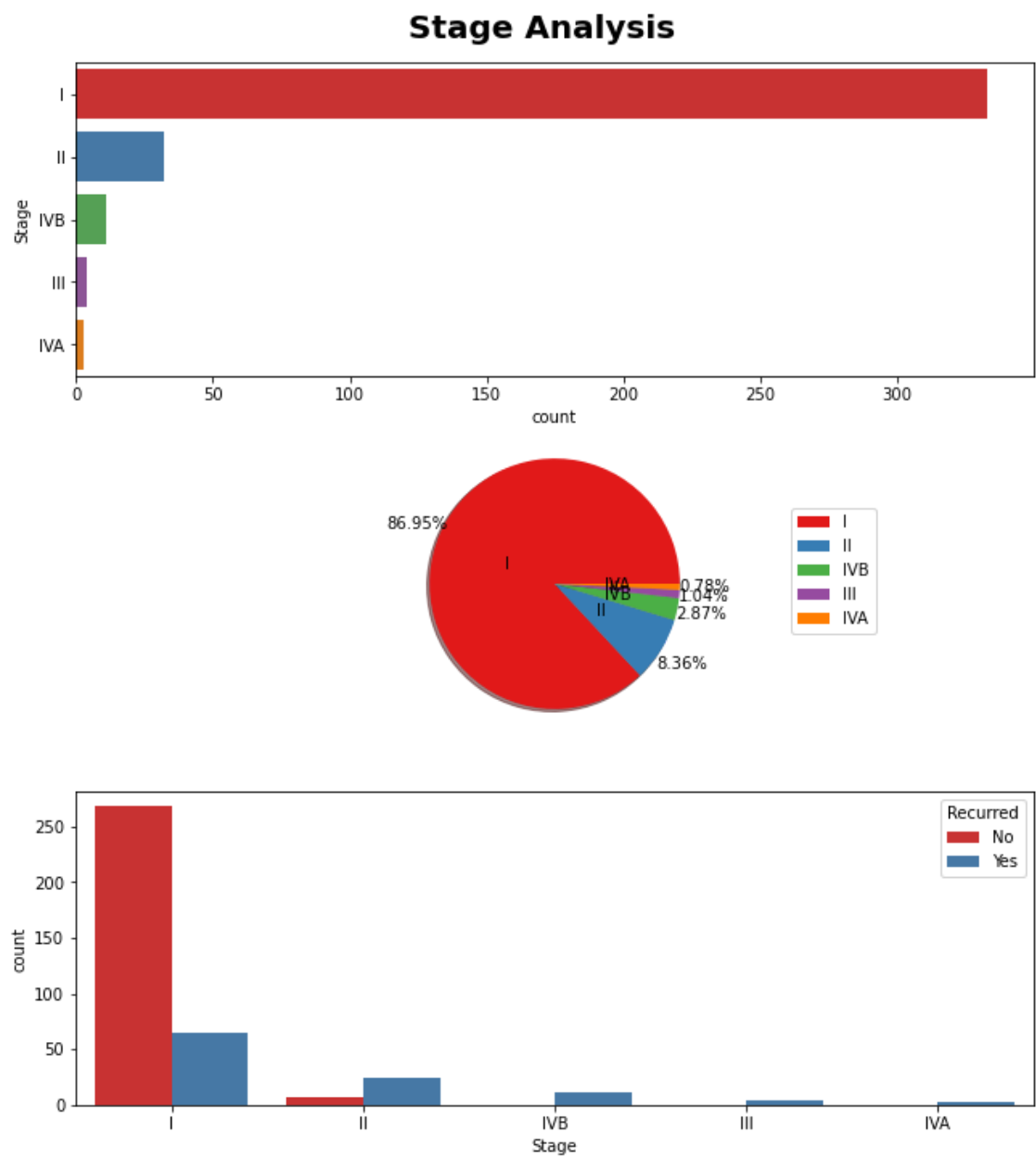
```
In [20]: ▶ sns.set_palette('Set1')
fig, ax = plt.subplots(3, 1, figsize=(9, 10))
fig.suptitle('Stage Analysis', fontsize=20, fontweight='bold')
plt.tight_layout()

count = df['Stage'].value_counts()

labels = df['Stage'].value_counts().index.tolist()

#Top ax
sns.countplot(y="Stage", data=df, ax=ax[0])
#middle ax
ax[1].pie(count, autopct='%.2f%%', labels=labels, shadow=True, pctdistance=0.8)
ax[1].legend(bbox_to_anchor=(1, 1), loc=2, borderaxespad=5)

# Bottom ax
sns.countplot(x='Stage', hue='Recurred', data=df, ax=ax[2])
plt.show()
```



Hx-Radiothreapy Analysis

```
In [21]: sns.set_palette('Set2')
fig, ax = plt.subplots(3, 1, figsize=(7, 8))
fig.suptitle('Hx-Radiothreapy Analysis', fontsize=20, fontweight='bold')
plt.tight_layout()

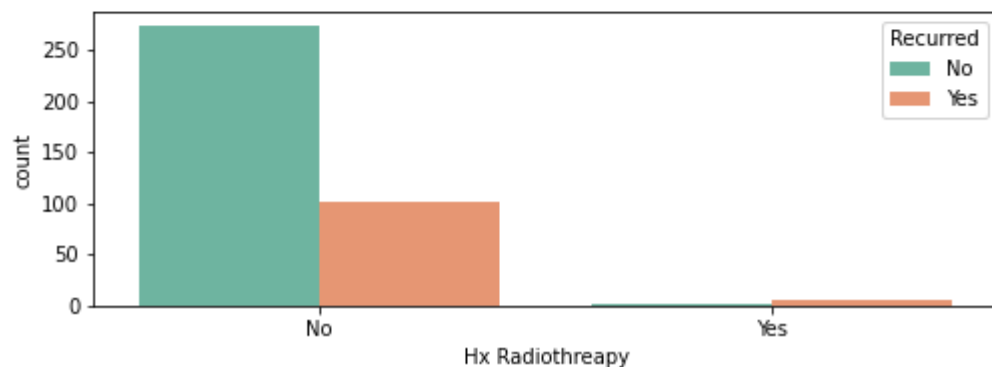
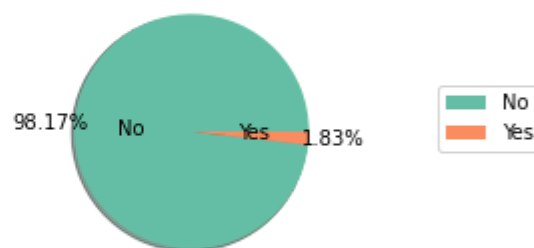
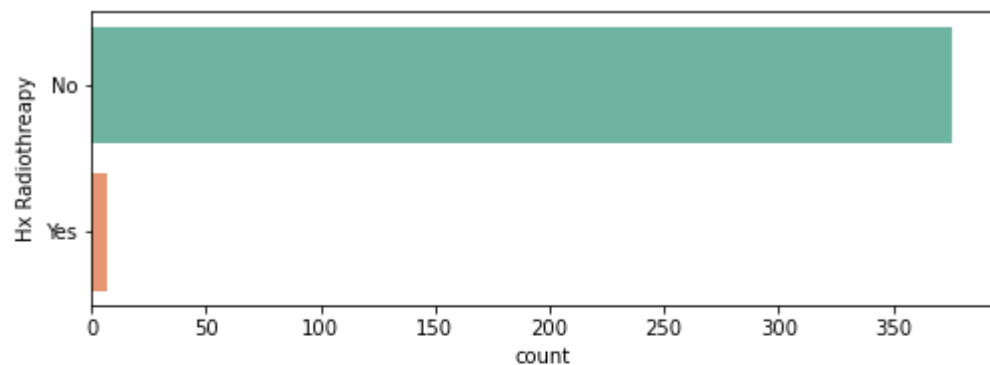
count = df['Hx Radiothreapy'].value_counts()

labels = df['Hx Radiothreapy'].value_counts().index.tolist()

#Top ax
sns.countplot(y="Hx Radiothreapy", data=df, ax=ax[0])
#middle ax
ax[1].pie(count, autopct='%0.2f%%', labels=labels, shadow=True, pctdistance=0.8)
ax[1].legend(bbox_to_anchor=(1, 1), loc=2, borderaxespad=5)

# Bottom ax
sns.countplot(x='Hx Radiothreapy', hue='Recurred', data=df, ax=ax[2])
plt.show()
```

Hx-Radiothreapy Analysis



Adenopathy Analysis

```
In [22]: ▶ sns.set_palette('Set1')
fig, ax = plt.subplots(3, 1, figsize=(6, 12))
fig.suptitle('Adenopathy Analysis', fontsize=20, fontweight='bold')
plt.tight_layout()

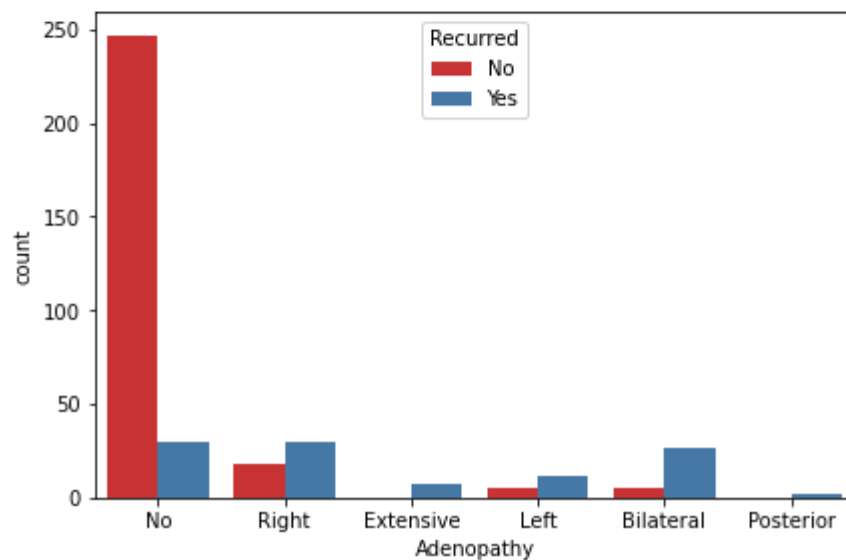
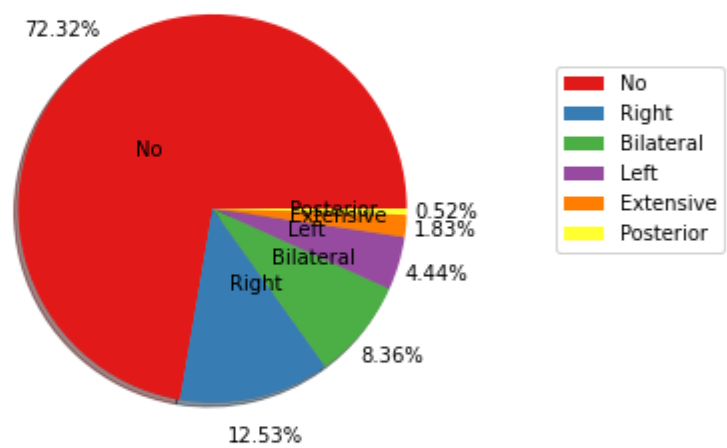
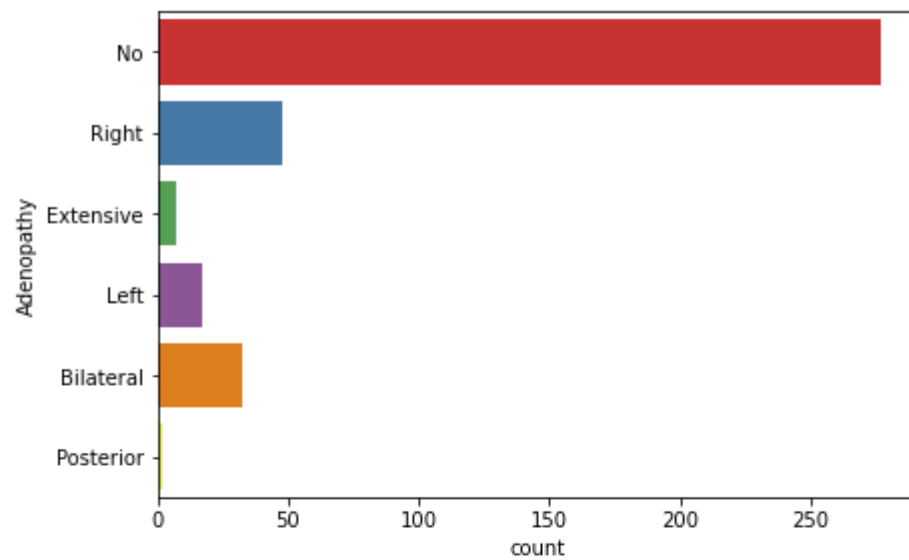
count = df.Adenopathy.value_counts()

labels = df.Adenopathy.value_counts().index.tolist()

#Top ax
sns.countplot(y="Adenopathy", data=df, ax=ax[0])
#middle ax
ax[1].pie(count, autopct='%.2f%%', labels=labels, shadow=True, pctdistance
ax[1].legend(bbox_to_anchor=(1, 1), loc=2, borderaxespad=5)

# Bottom ax
sns.countplot(x='Adenopathy', hue='Recurred', data=df, ax=ax[2])
plt.show()
```

Adenopathy Analysis



Pathology Analysis

```
In [23]: ▶ fig, ax = plt.subplots(3, 1, figsize=(6, 10))
fig.suptitle('Pathology Analysis', fontsize=20, fontweight='bold')
plt.tight_layout()

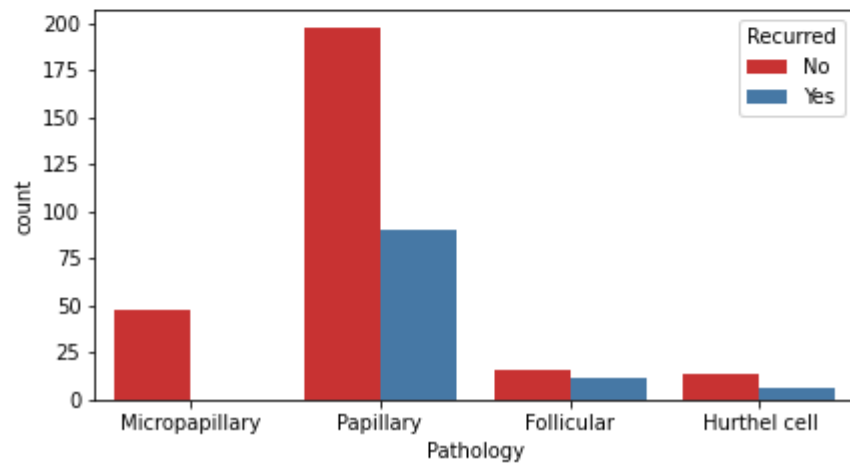
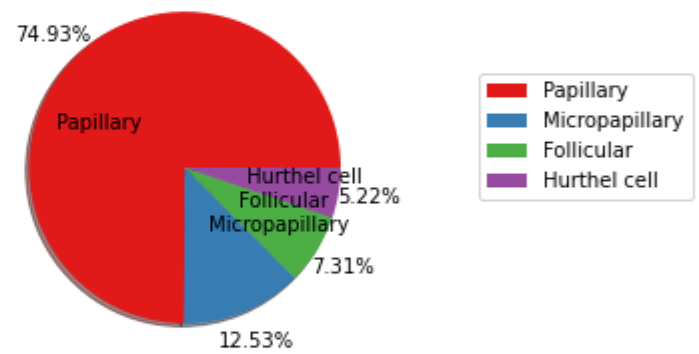
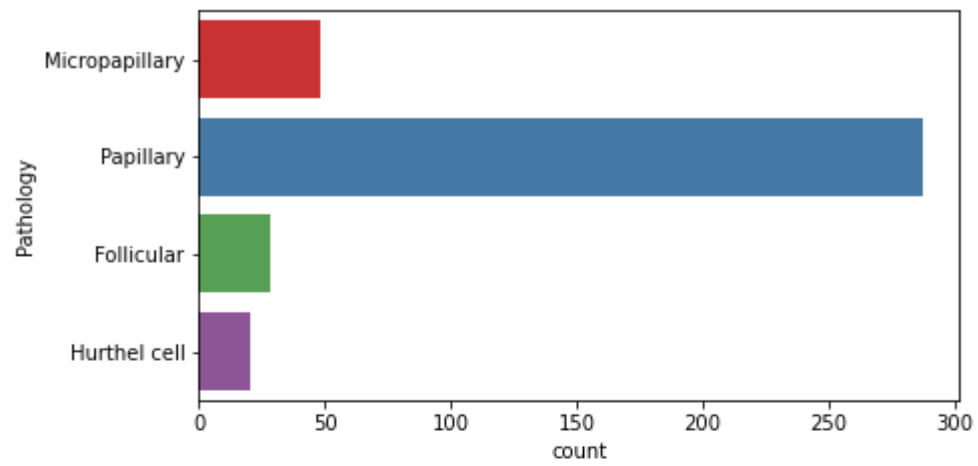
count = df.Pathology.value_counts()

labels = df.Pathology.value_counts().index.tolist()

#Top ax
sns.countplot(y="Pathology", data=df, ax=ax[0])
#middle ax
ax[1].pie(count, autopct='%.2f%%', labels=labels, shadow=True, pctdistance
ax[1].legend(bbox_to_anchor=(1, 1), loc=2, borderaxespad=5)

# Bottom ax
sns.countplot(x='Pathology', hue='Recurred', data=df, ax=ax[2])
plt.show()
```


Pathology Analysis



Physical Examination Analysis

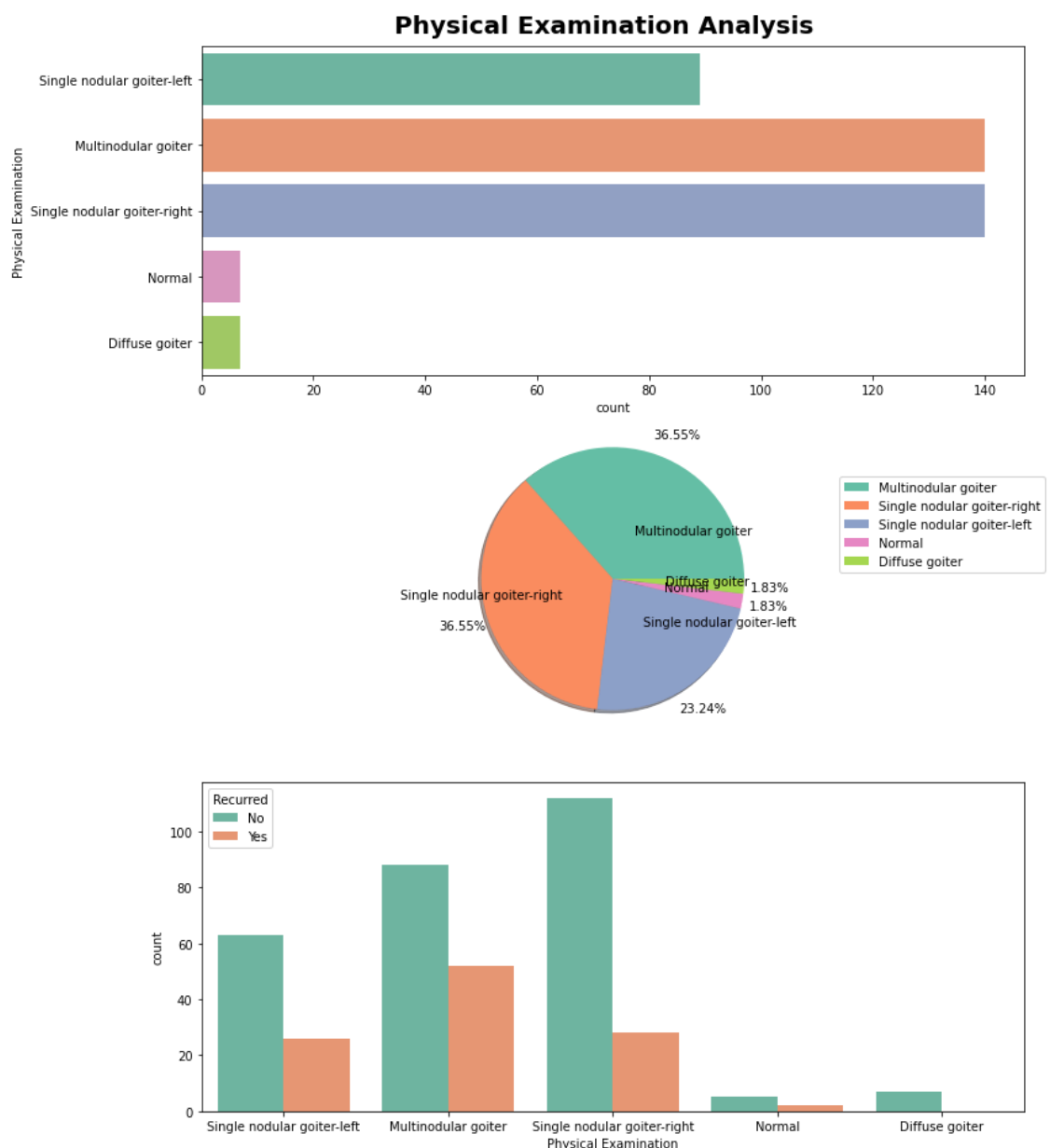
```
In [24]: sns.set_palette('Set2')
fig, ax = plt.subplots(3, 1, figsize=(10, 13))
fig.suptitle('Physical Examination Analysis', fontsize=20, fontweight='bold')
plt.tight_layout()

count = df['Physical Examination'].value_counts()

labels = df['Physical Examination'].value_counts().index.tolist()

#Top ax
sns.countplot(y="Physical Examination", data=df, ax=ax[0])
#Middle ax
ax[1].pie(count, autopct='%0.2f%', labels=labels, shadow=True, pctdistance=0.8)
ax[1].legend(bbox_to_anchor=(1, 1), loc=2, borderaxespad=5)

# Bottom ax
sns.countplot(x='Physical Examination', hue='Recurred', data=df, ax=ax[2])
plt.show()
```



Tumor Analysis

```
In [25]: ▶ sns.set_palette('Set2')
fig, ax = plt.subplots(3, 1, figsize=(8, 12))
fig.suptitle('Tumor Analysis', fontsize=20, fontweight='bold')
plt.tight_layout()

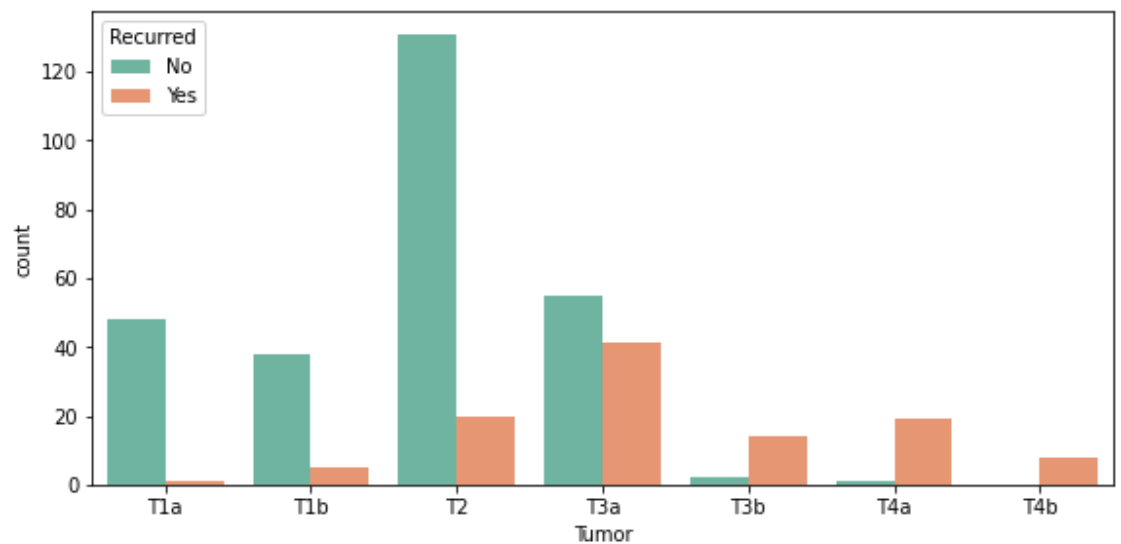
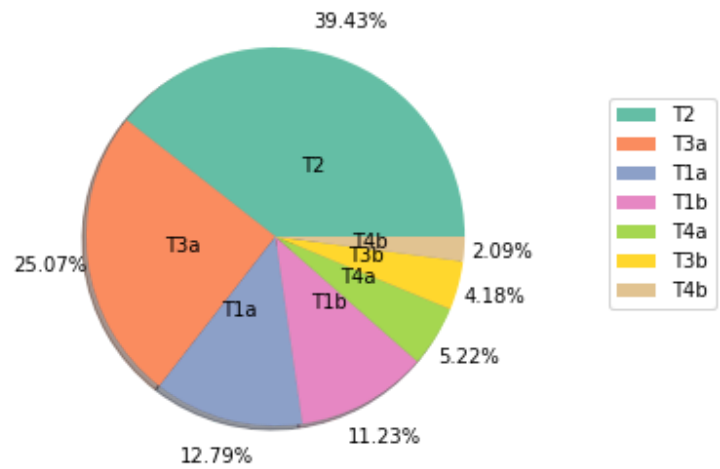
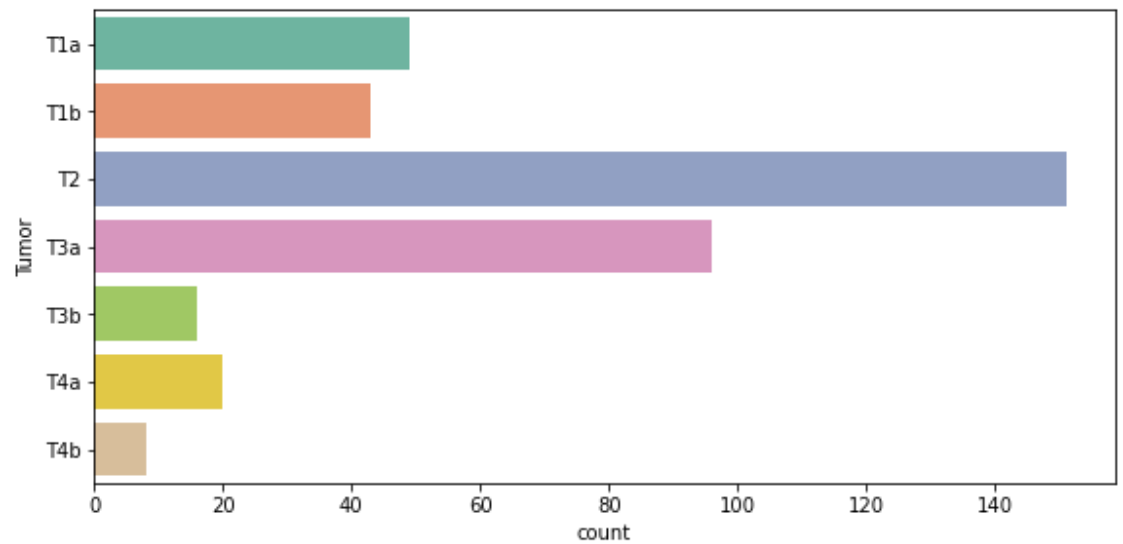
count = df.Tumor.value_counts()

labels = df.Tumor.value_counts().index.tolist()

#Top ax
sns.countplot(y="Tumor", data=df, ax=ax[0])
#middle ax
ax[1].pie(count, autopct='%.2f%%', labels=labels, shadow=True, pctdistance=0.8)
ax[1].legend(bbox_to_anchor=(1, 1), loc=2, borderaxespad=5)

# Bottom ax
sns.countplot(x='Tumor', hue='Recurred', data=df, ax=ax[2])
plt.show()
```

Tumor Analysis



Nodal Analysis

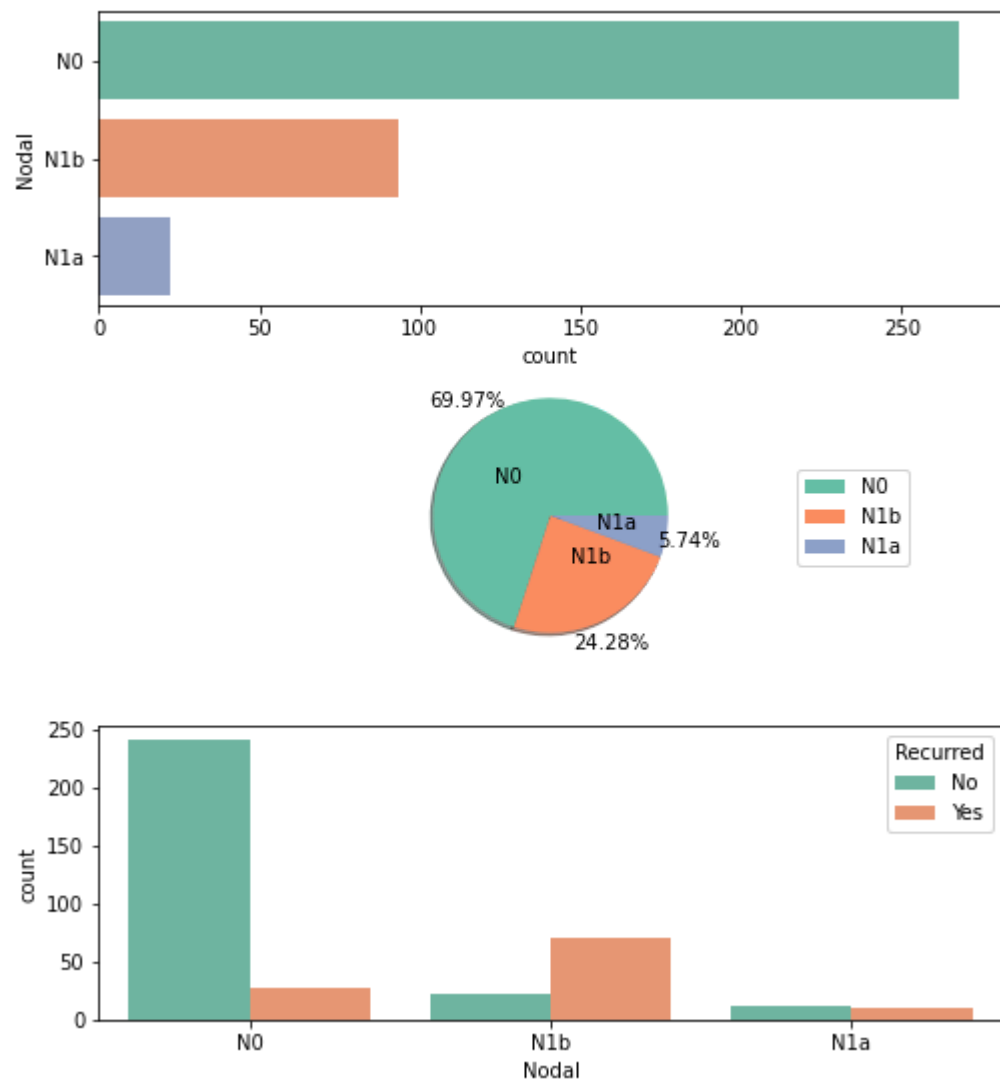
```
In [26]: sns.set_palette('Set2')
fig, ax = plt.subplots(3, 1, figsize=(7, 8))
fig.suptitle('Nodal Analysis', fontsize=20, fontweight='bold')
plt.tight_layout()

count = df.Nodal.value_counts()

labels = df.Nodal.value_counts().index.tolist()
#Top ax
sns.countplot(y="Nodal", data=df, ax=ax[0])
#Middle ax
ax[1].pie(count, autopct='%.2f%', labels=labels, shadow=True, pctdistance
ax[1].legend(bbox_to_anchor=(1, 1), loc=2, borderaxespad=5)

# Bottom ax
sns.countplot(x='Nodal', hue='Recurred', data=df, ax=ax[2])
plt.show()
```

Nodal Analysis



Metastasis Analysis

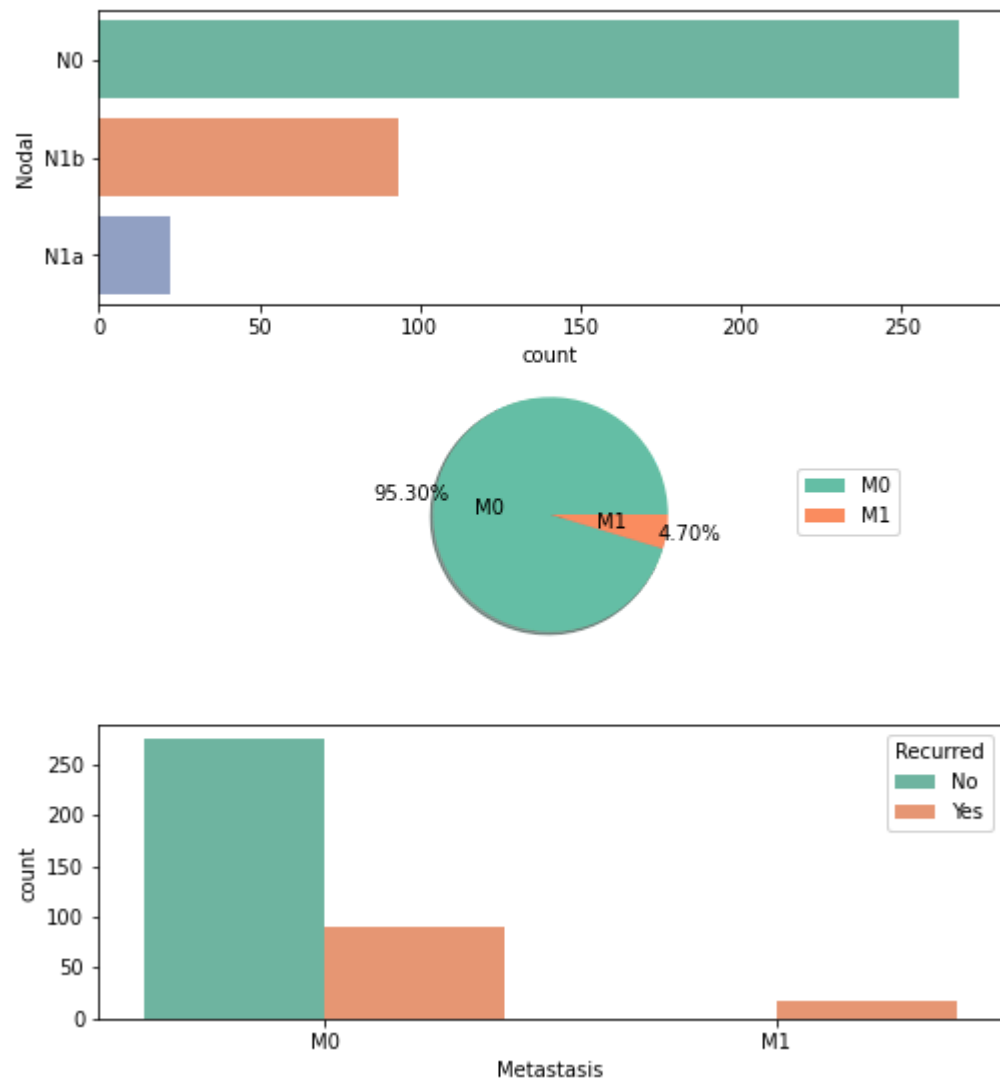
```
In [27]: sns.set_palette('Set2')
fig, ax = plt.subplots(3, 1, figsize=(7, 8))
fig.suptitle('Metastasis Analysis', fontsize=20, fontweight='bold')
plt.tight_layout()

count = df.Metastasis.value_counts()

labels = df.Metastasis.value_counts().index.tolist()
#Top ax
sns.countplot(y="Nodal", data=df, ax=ax[0])
#Middle ax
ax[1].pie(count, autopct='%0.2f%%', labels=labels, shadow=True, pctdistance=0.8)
ax[1].legend(bbox_to_anchor=(1, 1), loc=2, borderaxespad=5)

# Bottom ax
sns.countplot(x='Metastasis', hue='Recurred', data=df, ax=ax[2])
plt.show()
```

Metastasis Analysis



Data preprocessing

split

```
In [28]: ▶ inputs_df = df.drop('Recurred',axis=1)
         targets_df= df[['Recurred']]
```

preprocess categorical columns

```
In [29]: ▶ categorical_cols = inputs_df.select_dtypes(include='object').columns.tolist()
         categorical_cols
```

```
Out[29]: ['Gender',
          'Smoking',
          'Hx Smoking',
          'Hx Radiotherapy',
          'Thyroid Function',
          'Physical Examination',
          'Adenopathy',
          'Pathology',
          'Focality',
          'Risk',
          'Tumor',
          'Nodal',
          'Metastasis',
          'Stage',
          'Response']
```

```
In [32]: ▶ # preprocess the categorical
         encoder = OneHotEncoder(sparse=False)
         encoder.fit(inputs_df[categorical_cols])
         encoder_cols = encoder.get_feature_names_out(categorical_cols)
         inputs_df[encoder_cols] = encoder.transform(inputs_df[categorical_cols])
```

```
In [33]: ▶ final_df = pd.concat([inputs_df['Age'],inputs_df[encoder_cols]],axis=1)
```

preprocess numeric columns

```
In [34]: ▶ scaler = MinMaxScaler()
         final_df[['Age']] = scaler.fit_transform(final_df[['Age']])
```

```
In [35]: ▶ X = final_df # independent
         y= targets_df # dependent
```

In [36]: X

Out[36]:

	Age	Gender_F	Gender_M	Smoking_No	Smoking_Yes	Hx Smoking_No	H Smoking_Yes
0	0.179104	1.0	0.0	1.0	0.0	1.0	0
1	0.283582	1.0	0.0	1.0	0.0	0.0	1
2	0.223881	1.0	0.0	1.0	0.0	1.0	0
3	0.701493	1.0	0.0	1.0	0.0	1.0	0
4	0.701493	1.0	0.0	1.0	0.0	1.0	0
...	...	...	...	...	...	...	...
378	0.850746	0.0	1.0	0.0	1.0	0.0	1
379	0.985075	0.0	1.0	0.0	1.0	1.0	0
380	0.850746	0.0	1.0	0.0	1.0	0.0	1
381	0.686567	0.0	1.0	0.0	1.0	0.0	1
382	0.776119	0.0	1.0	0.0	1.0	1.0	0

383 rows × 55 columns



In [37]: y

Out[37]:

	Recurred
0	No
1	No
2	No
3	No
4	No
...	...
378	Yes
379	Yes
380	Yes
381	Yes
382	Yes

383 rows × 1 columns

Modeling

cross-validation

```
In [38]: xtrain,xtest,ytrain,ytest = train_test_split(X,y,test_size=.20,random_stat
```

Analysis a model performance

```
In [39]: logist = LogisticRegression()
logist.fit(xtrain,ytrain)
```

C:\Users\jemin kamani\anaconda3\lib\site-packages\sklearn\utils\validation.py:1111: DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
Out[39]: LogisticRegression
LogisticRegression()
```

```
In [40]: logist.predict(xtest)
```

```
Out[40]: array(['No', 'No', 'Yes', 'Yes', 'No', 'Yes', 'No', 'No', 'Yes', 'Yes',
                'No', 'No', 'No', 'No', 'No', 'No', 'No', 'Yes', 'Yes', 'No', 'N
                o',
                'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'Yes', 'No', 'No',
                'Yes', 'No', 'No', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'No',
                'No', 'No', 'No', 'No', 'No', 'No', 'Yes', 'No', 'No', 'No', 'Ye
                s',
                'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No',
                'Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'No', 'No', 'No', 'No',
                'No', 'Yes', 'Yes'], dtype=object)
```

```
In [41]: print(classification_report(ytest,logist.predict(xtest)))
```

	precision	recall	f1-score	support
No	0.98	0.98	0.98	58
Yes	0.95	0.95	0.95	19
accuracy			0.97	77
macro avg	0.97	0.97	0.97	77
weighted avg	0.97	0.97	0.97	77

```
In [42]: cm = confusion_matrix(ytest,logist.predict(xtest))
cm
```

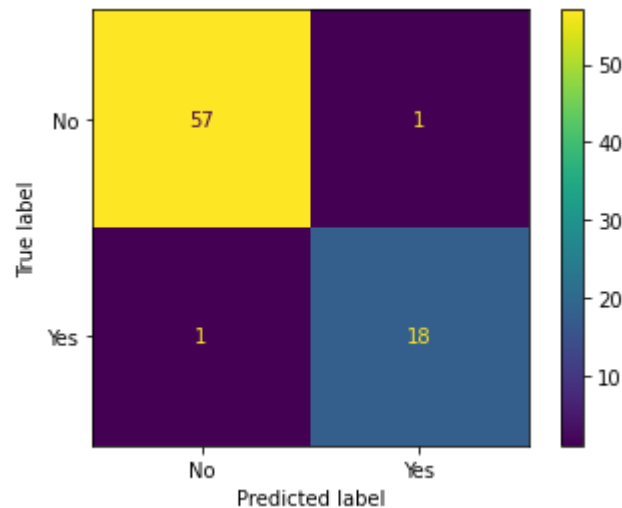
```
Out[42]: array([[57,  1],
                [ 1, 18]], dtype=int64)
```

```
In [43]: accuracy_score(ytest,logist.predict(xtest))
```

```
Out[43]: 0.974025974025974
```

```
In [44]: fig = ConfusionMatrixDisplay(cm,display_labels=logist.classes_)
fig.plot()
```

```
Out[44]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1dfb1eb2b20>
```



choose our best model for this dataset

```
In [45]: logist = LogisticRegression()
sgd = SGDClassifier()
svc = SVC()
random = RandomForestClassifier()
grad = GradientBoostingClassifier()
bag = BaggingClassifier()
```

```
In [46]: model_li = [logist,sgd,svc,random,grad,bag]
model_li
```

```
Out[46]: [LogisticRegression(),
SGDClassifier(),
SVC(),
RandomForestClassifier(),
GradientBoostingClassifier(),
BaggingClassifier()]
```

```
In [47]: accuracy = []  
for model in model_li:  
    model.fit(xtrain,ytrain)  
    accuracy.append(accuracy_score(ytest,model.predict(xtest)))
```

C:\Users\jemin kamani\anaconda3\lib\site-packages\sklearn\utils\validation.py:1111: DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

C:\Users\jemin kamani\anaconda3\lib\site-packages\sklearn\utils\validation.py:1111: DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

C:\Users\jemin kamani\anaconda3\lib\site-packages\sklearn\utils\validation.py:1111: DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

C:\Users\jemin kamani\AppData\Local\Temp\ipykernel_20956\1828380418.py:3: DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

C:\Users\jemin kamani\anaconda3\lib\site-packages\sklearn\ensemble_gb.py:570: DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

C:\Users\jemin kamani\anaconda3\lib\site-packages\sklearn\ensemble_bagging.py:760: DataConversionWarning:

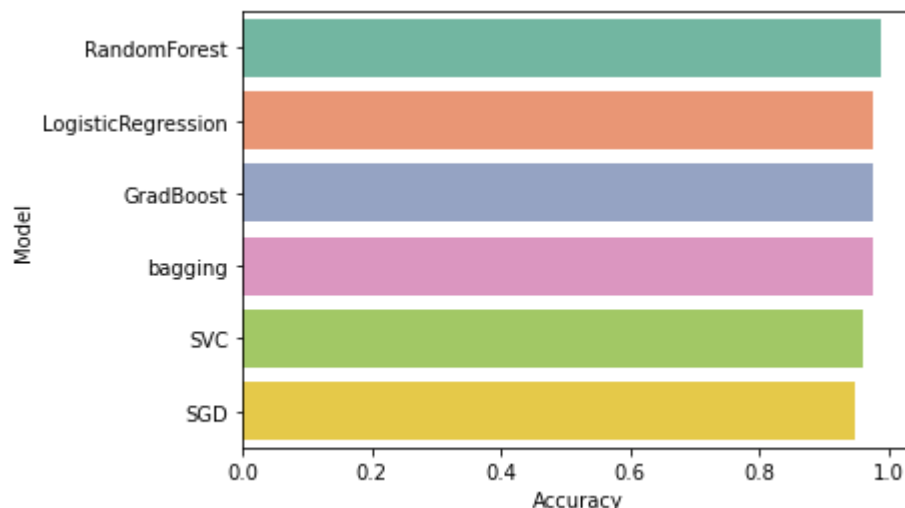
A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
In [48]: accuracy
```

```
Out[48]: [0.974025974025974,  
          0.948051948051948,  
          0.961038961038961,  
          0.987012987012987,  
          0.974025974025974,  
          0.974025974025974]
```

```
In [49]: ▶ model_result = {
            'Model': ["LogisticRegression", "SGD", "SVC", 'RandomForest', 'GradBoost', '
            'Accuracy': accuracy
        }
        result_df = pd.DataFrame(model_result).sort_values(by='Accuracy', ascending
        sns.barplot(data=result_df, y='Model', x='Accuracy')
```

Out[49]: <AxesSubplot:xlabel='Accuracy', ylabel='Model'>



Optimize our Final Model

```
In [50]: ▶ def test_params(**params):
            model = RandomForestClassifier(random_state=42, n_jobs=-1, **params)
            model.fit(xtrain, ytrain)
            accuracy = accuracy_score(ytest, model.predict(xtest))
            print("The accuracy is : ", accuracy)
```

```
In [51]: ▶ test_params(min_samples_split=15)
```

The accuracy is : 0.987012987012987

C:\Users\jemin kamani\AppData\Local\Temp\ipykernel_20956\902571088.py:3:
DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
In [52]: ▶ test_params(max_depth=15)
```

C:\Users\jemin kamani\AppData\Local\Temp\ipykernel_20956\902571088.py:3:
DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

The accuracy is : 0.974025974025974

In [53]: `test_params(min_samples_split=5,max_depth=16)`

C:\Users\jemin kamani\AppData\Local\Temp\ipykernel_20956\902571088.py:3:
DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

The accuracy is : 0.987012987012987

In [54]: `test_params(min_weight_fraction_leaf=0.1)`

C:\Users\jemin kamani\AppData\Local\Temp\ipykernel_20956\902571088.py:3:
DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

The accuracy is : 0.922077922077922

In [55]: `test_params(min_samples_split=5,max_depth=10)`

C:\Users\jemin kamani\AppData\Local\Temp\ipykernel_20956\902571088.py:3:
DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

The accuracy is : 0.987012987012987

In [56]: `test_params(min_impurity_decrease=0.1)`

C:\Users\jemin kamani\AppData\Local\Temp\ipykernel_20956\902571088.py:3:
DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

The accuracy is : 0.922077922077922

Final model with parameter and performance

```
In [57]: # final model
random = RandomForestClassifier(random_state=42,min_samples_split=5,max_de
random.fit(xtrain,ytrain)
accuracy = accuracy_score(ytest,random.predict(xtest))
print("The accuracy is : ",accuracy)
```

C:\Users\jemin kamani\AppData\Local\Temp\ipykernel_20956\835013509.py:3:
DataConversionWarning:

A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

The accuracy is : 0.987012987012987

```
In [58]: print(classification_report(ytest,random.predict(xtest)))
```

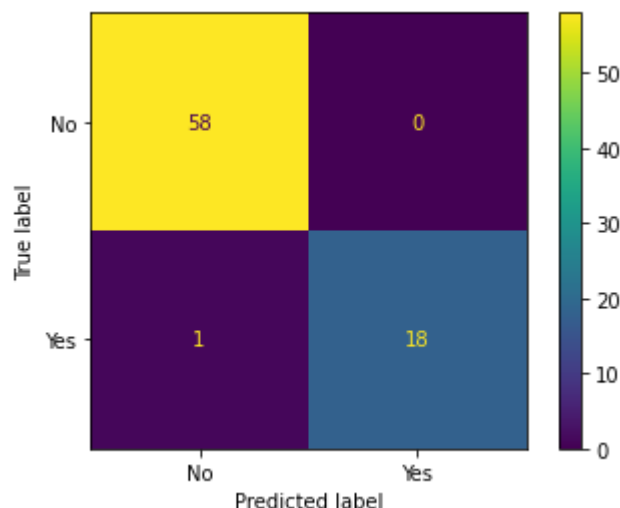
	precision	recall	f1-score	support
No	0.98	1.00	0.99	58
Yes	1.00	0.95	0.97	19
accuracy			0.99	77
macro avg	0.99	0.97	0.98	77
weighted avg	0.99	0.99	0.99	77

```
In [59]: cm = confusion_matrix(ytest,random.predict(xtest))
cm
```

```
Out[59]: array([[58,  0],
               [ 1, 18]], dtype=int64)
```

```
In [60]: fig = ConfusionMatrixDisplay(cm,display_labels=logist.classes_)
fig.plot()
```

```
Out[60]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1dfb5
524040>
```



Create a single output

create a single input and predict stepwise

```
In [61]: new_input = {
    'Age': 30,
    'Gender': 'F',
    'Smoking': 'No',
    'Hx Smoking': 'No',
    'Hx Radiothreapy': 'No',
    'Thyroid Function': 'Euthyroid',
    'Physical Examination': 'Multinodular goiter',
    'Adenopathy': 'No',
    'Pathology': 'Micropapillary',
    'Focality': 'Uni-Focal',
    'Risk': 'Low',
    'Tumor': 'T1a',
    'Nodal': 'N0',
    'Metastasis': 'M0',
    'Stage': 'I',
    'Response': 'Excellent'
}
```

```
In [62]: new_df = pd.DataFrame([new_input])
new_df
```

Out[62]:

	Age	Gender	Smoking	Hx Smoking	Hx Radiothreapy	Thyroid Function	Physical Examination	Adenopathy	
0	30	F	No	No	No	Euthyroid	Multinodular goiter	No	Mi

```
In [63]: new_df.shape
```

Out[63]: (1, 16)

```
In [64]: new_df[encoder_cols] = encoder.transform(new_df[categorical_cols])
new_df[['Age']] = scaler.fit_transform(new_df[['Age']])
new_df = pd.concat([new_df[['Age']], new_df[encoder_cols]], axis=1)
```

```
In [65]: new_df
```

Out[65]:

	Age	Gender_F	Gender_M	Smoking_No	Smoking_Yes	Hx Smoking_No	Hx Smoking_Yes	Rac
0	0.0	1.0	0.0	1.0	0.0	1.0	0.0	

1 rows × 55 columns

```
In [66]: random.predict(new_df)
```

```
Out[66]: array(['No'], dtype=object)
```

create single predict function

```
In [67]: def single_prediction(new_input):
          new_df = pd.DataFrame([new_input])
          new_df[encoder_cols] = encoder.transform(new_df[categorical_cols])
          new_df[['Age']] = scaler.fit_transform(new_df[['Age']])
          new_df = pd.concat([new_df[['Age']], new_df[encoder_cols]], axis=1)
          pred = random.predict(new_df)[0]
          proba = random.predict_proba(new_df)[0][0]
          print('Single value prediction is : ', pred)
          print('Single value predic probability is : ', proba)
```

```
In [68]: single_prediction(new_input)
```

```
Single value prediction is : No
Single value predic probability is : 0.97875
```

Save and Load the model

save

```
In [69]: model_part = {
          'model': random
        }
```

```
In [70]: joblib.dump(model_part, 'Thyroid model.joblib')
```

```
Out[70]: ['Thyroid model.joblib']
```

load

```
In [71]: # Load the model
          random = joblib.load('Thyroid model.joblib')
```


performance

```
In [72]: ▶ pred = random['model'].predict(xtest)
pred
```

```
Out[72]: array(['No', 'No', 'Yes', 'Yes', 'No', 'Yes', 'No', 'No', 'Yes', 'Yes',
                'No', 'No', 'No', 'No', 'No', 'No', 'No', 'Yes', 'Yes', 'No', 'N
o',
                'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'Yes', 'No', 'No',
                'Yes', 'No', 'No', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'No',
                'No', 'No', 'No', 'No', 'No', 'No', 'Yes', 'No', 'No', 'No', 'Ye
s',
                'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'No',
                'Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'No', 'No', 'No', 'No',
                'No', 'Yes', 'No'], dtype=object)
```

```
In [73]: ▶ proba = random['model'].predict_proba(xtest)
proba
```

```
Out[73]: array([[1.          , 0.          ],
 [0.96191667, 0.03808333],
 [0.04402381, 0.95597619],
 [0.28420238, 0.71579762],
 [1.          , 0.          ],
 [0.08691667, 0.91308333],
 [1.          , 0.          ],
 [1.          , 0.          ],
 [0.          , 1.          ],
 [0.47         , 0.53         ],
 [0.73602381, 0.26397619],
 [0.99866667, 0.00133333],
 [0.95184524, 0.04815476],
 [0.95838095, 0.04161905],
 [0.99866667, 0.00133333],
 [1.          , 0.          ],
 [0.99866667, 0.00133333],
 [0.03045455, 0.96954545],
 [0.0425       , 0.9575       ],
 [0.98114286, 0.01885714],
 [0.97875      , 0.02125      ],
 [1.          , 0.          ],
 [1.          , 0.          ],
 [0.97797619, 0.02202381],
 [0.99166667, 0.00833333],
 [0.887        , 0.113        ],
 [0.99866667, 0.00133333],
 [0.84383333, 0.15616667],
 [0.99         , 0.01         ],
 [0.35816667, 0.64183333],
 [0.98666667, 0.01333333],
 [1.          , 0.          ],
 [0.304        , 0.696        ],
 [1.          , 0.          ],
 [0.99866667, 0.00133333],
 [0.44880952, 0.55119048],
 [0.99666667, 0.00333333],
 [0.93835714, 0.06164286],
 [0.05709524, 0.94290476],
 [0.04402381, 0.95597619],
 [1.          , 0.          ],
 [0.99866667, 0.00133333],
 [1.          , 0.          ],
 [0.99866667, 0.00133333],
 [0.96254762, 0.03745238],
 [1.          , 0.          ],
 [1.          , 0.          ],
 [1.          , 0.          ],
 [0.07116667, 0.92883333],
 [1.          , 0.          ],
 [1.          , 0.          ],
 [0.9975       , 0.0025       ],
 [0.0895       , 0.9105       ],
 [0.99875      , 0.00125      ],
 [0.99366667, 0.00633333],
 [0.87482143, 0.12517857],
 [0.8577619   , 0.1422381   ],
 [1.          , 0.          ],
 [0.85839286, 0.14160714],
 [1.          , 0.          ],
 [1.          , 0.          ]])
```

```
[0.97216667, 0.02783333],  
[0.995      , 0.005      ],  
[1.         , 0.         ],  
[0.10664286, 0.89335714],  
[0.87556349, 0.12443651],  
[0.11775    , 0.88225    ],  
[1.         , 0.         ],  
[0.00571429, 0.99428571],  
[1.         , 0.         ],  
[1.         , 0.         ],  
[0.99866667, 0.00133333],  
[0.97791667, 0.02208333],  
[0.9825     , 0.0175     ],  
[0.84005952, 0.15994048],  
[0.12416667, 0.87583333],  
[0.6455     , 0.3545     ]])
```